



# Programmiersvorkurs

---

**Einführung in C# - Tag 2**

# Inhaltsübersicht



|              |                                                                                |
|--------------|--------------------------------------------------------------------------------|
| <b>Tag 1</b> | Variablen, Datentypen, Arithmetik, Konvertierung, Zustände, Visual Studio Demo |
| <b>Tag 2</b> | Boolsche Ausdrücke, Kommentare, If-Abfragen, Switch-Case                       |
| <b>Tag 3</b> | Arrays, (Do-)While-Schleife, For-Schleifen, Debugging                          |
| <b>Tag 4</b> | Methoden, Klassen                                                              |

# Arrays

- Ein Array fasst mehrere Variablen des gleichen Typs zusammen.
- Alle Werte müssen vom gleichen Typ sein.

```
static void Main(string[] args)
{
    int[] zahlen = { 4, 8, 16, 32, 1, 9 };

    string[] wörter = { "Hallo", "Arrays", "Texte", "Wörter" };

    double[] schwierig = { "falsch", "nein", 21, 54.2222, 'c' };
}
```

# Arrays erstellen

- Um ein Array vom Typ `type` zu deklarieren

```
Type[] arrayName;
```

- Um ein Array vom Typ `type` und Größe `n` zu deklarieren und initialisieren.
- Das Array wird mit Standardwerten gefüllt (Bsp. `Int` → 0)

```
Type[] arrayName = new Type[n];
```

# Arrays erstellen

Um ein Array mit Werten zu initialisieren:

```
int[] arrayName = new int[] { 15, 583 };
```

Geht auch kürzer:

```
int[] arrayName = { 15, 583 };
```

Größen von Arrays sind nicht veränderbar.

# Arrayzugriff

- Zugriff auf das i-te Arrayelement:
- Achtung: Der Index beginnt bei 0 und endet bei der [Größe]-1

```
arrayName[i]
```

Die Größe des Arrays kann mit der Funktion Length ermittelt werden

```
arrayName.Length
```

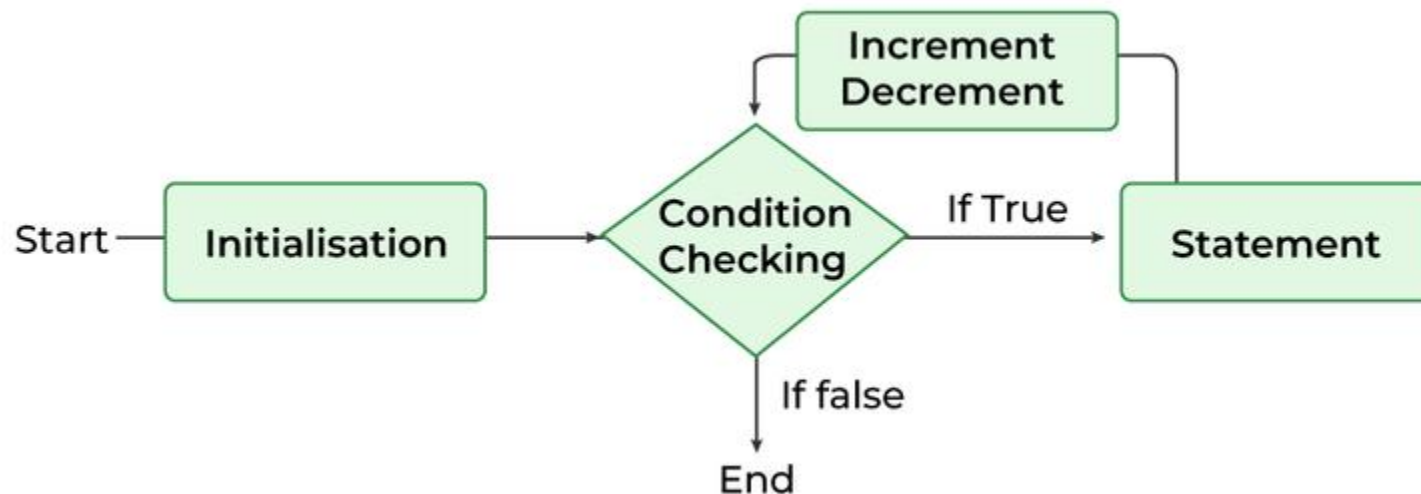
# Arrayzugriff

```
int[] arrayInt = { 1, 2, 3, 4, 5, 6 };  
Console.WriteLine(arrayInt[2]);  
//Output: 3  
Console.WriteLine(arrayInt.Length);  
//Output: 6
```

- Vorsicht beim Zugriff: `OutOfRangeException`

# Schleifen

- Eine Struktur, um Anweisungen wiederholt auszuführen
- *Zählschleifen – for*
- *Bedingungsgeprüft – while*
- *Element für Element – foreach*



# While-Schleifen

- while ist perfekt, wenn nicht klar ist, wie oft die Schleife läuft.
- Bedingungen sind dynamisch (Benutzereingabe, Werte, etc.).

```
while (Bedingung)
{
    Anweisung1;
    Anweisung2;
    //...
}
```

```
Boolean isHappy = true;

while (isHappy == true)
{
    Console.WriteLine("Hello World");

    //RandomEreignis
    isHappy == false;
}
```

# Do-While-Schleifen

- While, aber Andersrum. Es wird zuerst ausgeführt, dann die Bedingung überprüft.
- Also mindestens einmal durchlaufen.

```
do
{
    Anweisung1;
    Anweisung2;
    //...
} while (Bedingung);
```

```
int zaehler = 10;

do
{
    Console.WriteLine("Hello World");
    zaehler++;
} while (zaehler < 10);
```

# Endlosschleifen

---

```
int counter = 0;

while (true)
{
    Console.Write(counter++);
}
```

# For-Schleifen

---

- Initialisierung
- Bedingung wird geprüft (Wahr oder falsch)
- Schritt für Schritt Anweisungen
- Schritt (bspw.  $i+1$ )

```
for (Initialisierung; Bedingung; Schritt)
{
    Anweisung1;
    Anweisung2;
}
```

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine("Hallo");
}
```

# For-Schleifen + Arrays

---

```
char[] abc = { 'a', 'b', 'c' };  
for (int i = 0; i < abc.Length; i++)  
{  
    Console.WriteLine(abc[i]);  
}
```

# Nested Schleifen

---

```
for (int i = 1; i <= 5; i++) // Zeilen
{
    for (int j = 1; j <= i; j++) // Sterne pro Zeile
    {
        Console.Write("*");
    }
    Console.WriteLine();
}
```

# Wann welche Schleifen

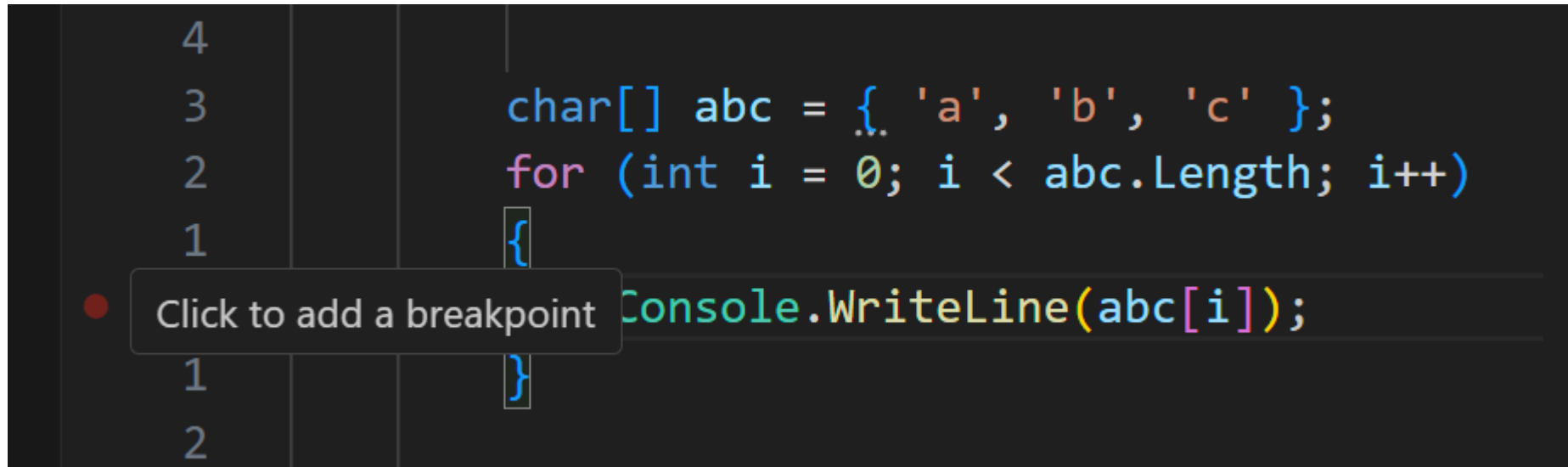
---



- For–Schleife
  - Wenn man eine bestimmte Anzahl an Durchläufen haben möchte
- While–Schleife
  - Wenn man nur eine Bedingung prüft, ohne bestimmter Anzahl

Breakpoints: Ein Stopp bei einem Durchlauf, um einen aktuellen Zustand zu sehen.

- Doppelklick auf den Zeilenrand
- F5 zum Debugger starten



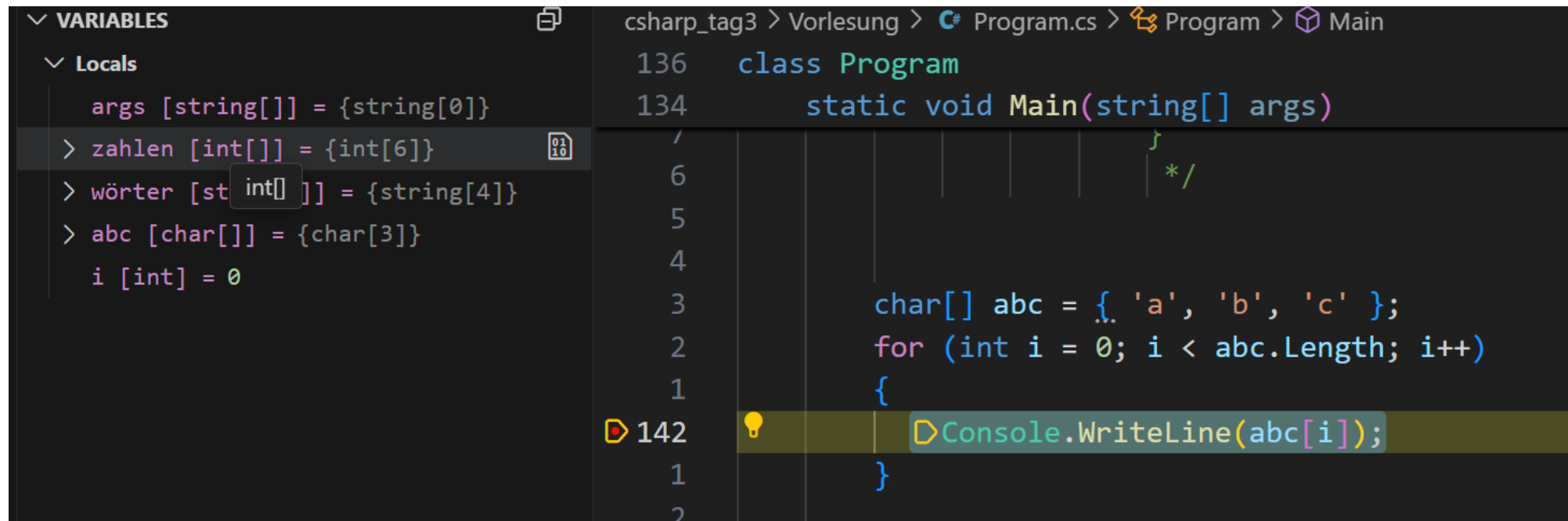
```
4  
3 char[] abc = { 'a', 'b', 'c' };  
2 for (int i = 0; i < abc.Length; i++)  
1 {  
  Console.WriteLine(abc[i]);  
1 }  
2
```

Click to add a breakpoint

# Debugging

Breakpoints: Ein Stopp bei einem Durchlauf, um einen aktuellen Zustand zu sehen.

- Doppelklick auf den Zeilenrand
- F5 zum Debugger starten



The screenshot shows the Visual Studio IDE with a C# program open. The left sidebar displays the 'VARIABLES' window, which is expanded to show 'Locals'. The variables listed are: `args [string[]] = {string[0]}`, `> zahlen [int[]] = {int[6]}` (highlighted), `> wörter [string[]] = {string[4]}`, `> abc [char[]] = {char[3]}`, and `i [int] = 0`. The main editor shows the code for `class Program` with a `static void Main(string[] args)` method. A breakpoint (yellow lightbulb icon) is set on line 142, which is `Console.WriteLine(abc[i]);`. The code also includes a comment `/*` on line 6 and a character array `char[] abc = { 'a', 'b', 'c' };` on line 3.

```
cssharp_tag3 > Vorlesung > C# Program.cs > Program > Main
136 class Program
134 static void Main(string[] args)
/
6
5
4
3 char[] abc = { 'a', 'b', 'c' };
2 for (int i = 0; i < abc.Length; i++)
1 {
142 Console.WriteLine(abc[i]);
1
2 }
```

# Quellen und Mitwirkende

---



Original 2013 Felix Bürkle

Überarbeitet 2015 Eva Wolkwitz

Überarbeitet 2016 Dominik Hartfelder

Überarbeitet 2016 Giulia Maier

Überarbeitet 2017 Johannes Beierle

Überarbeitet 2018 Giulia Maier

Überarbeitet 2021 Tobias Hnyk

Überarbeitet 2025 Miguel Rasch